

# AI/ML System Design Appendix

Production architectures, diagrams and trade-offs

Prepared for Pune / Remote India AI-ML Engineering interviews

Basic + practical + production-grade knowledge in one structured set

## How to use this PDF

Read for understanding first, then revise using the checklists and interview answer frames.

For each concept, practice saying: problem, data, model, evaluation, deployment, monitoring, failure handling.

This is written to reduce bookish answers and build production language for interviews.

## How to Use This Appendix

For each design, practice explaining requirements, data flow, model choice, APIs, monitoring, risks and trade-offs. Do not memorize the diagram. Memorize the reasoning.

# 1. ML Prediction API

Serve a trained classifier/regressor behind a typed API with monitoring and rollback.

## High-level architecture



| Design area                 | Production decision                                                        |
|-----------------------------|----------------------------------------------------------------------------|
| Functional requirements     | Define user action, input format, expected output and acceptance criteria. |
| Non-functional requirements | Latency, throughput, availability, cost, security and compliance.          |
| Data layer                  | Validate schema, freshness, duplicates, access control and lineage.        |
| Model/LLM layer             | Start with a baseline and justify model size, latency and cost.            |
| Evaluation                  | Use offline metrics, golden examples and online feedback where possible.   |
| Monitoring                  | Track quality, latency, errors, drift, token usage and user feedback.      |
| Failure handling            | Use retries, fallbacks, abstention, human escalation and rollback.         |

## Interview explanation frame

- I would start ml prediction api by clarifying the users, expected volume and risk level.
- I would define the data contract before choosing the model.
- I would build a baseline, measure it, then add complexity only if required.
- I would ship with logging, evaluation, monitoring and a rollback path.

## 2. Churn Prediction System

Predict customers likely to churn and feed retention actions.

### High-level architecture



| Design area                 | Production decision                                                        |
|-----------------------------|----------------------------------------------------------------------------|
| Functional requirements     | Define user action, input format, expected output and acceptance criteria. |
| Non-functional requirements | Latency, throughput, availability, cost, security and compliance.          |
| Data layer                  | Validate schema, freshness, duplicates, access control and lineage.        |
| Model/LLM layer             | Start with a baseline and justify model size, latency and cost.            |
| Evaluation                  | Use offline metrics, golden examples and online feedback where possible.   |
| Monitoring                  | Track quality, latency, errors, drift, token usage and user feedback.      |
| Failure handling            | Use retries, fallbacks, abstention, human escalation and rollback.         |

### Interview explanation frame

- I would start churn prediction system by clarifying the users, expected volume and risk level.
- I would define the data contract before choosing the model.
- I would build a baseline, measure it, then add complexity only if required.
- I would ship with logging, evaluation, monitoring and a rollback path.

### 3. Credit/Risk Scoring

Combine model scores with policy rules and explainability.

#### High-level architecture



| Design area                 | Production decision                                                        |
|-----------------------------|----------------------------------------------------------------------------|
| Functional requirements     | Define user action, input format, expected output and acceptance criteria. |
| Non-functional requirements | Latency, throughput, availability, cost, security and compliance.          |
| Data layer                  | Validate schema, freshness, duplicates, access control and lineage.        |
| Model/LLM layer             | Start with a baseline and justify model size, latency and cost.            |
| Evaluation                  | Use offline metrics, golden examples and online feedback where possible.   |
| Monitoring                  | Track quality, latency, errors, drift, token usage and user feedback.      |
| Failure handling            | Use retries, fallbacks, abstention, human escalation and rollback.         |

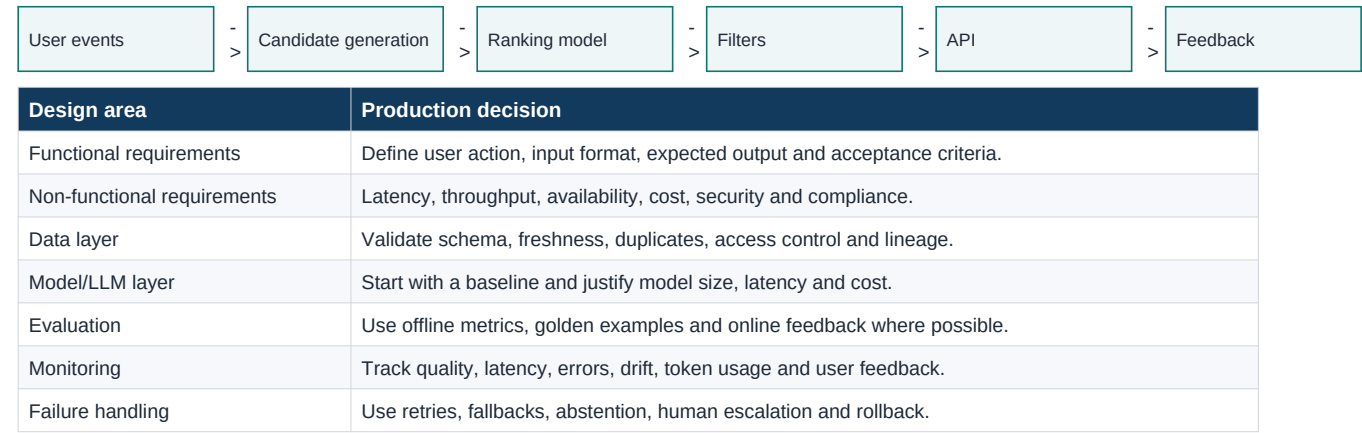
#### Interview explanation frame

- I would start credit/risk scoring by clarifying the users, expected volume and risk level.
- I would define the data contract before choosing the model.
- I would build a baseline, measure it, then add complexity only if required.
- I would ship with logging, evaluation, monitoring and a rollback path.

# 4. Recommendation System

Recommend products/content with freshness and business constraints.

## High-level architecture



## Interview explanation frame

- I would start recommendation system by clarifying the users, expected volume and risk level.
- I would define the data contract before choosing the model.
- I would build a baseline, measure it, then add complexity only if required.
- I would ship with logging, evaluation, monitoring and a rollback path.

# 5. Fraud Detection

Detect suspicious events with low latency and human review.

## High-level architecture



| Design area                 | Production decision                                                        |
|-----------------------------|----------------------------------------------------------------------------|
| Functional requirements     | Define user action, input format, expected output and acceptance criteria. |
| Non-functional requirements | Latency, throughput, availability, cost, security and compliance.          |
| Data layer                  | Validate schema, freshness, duplicates, access control and lineage.        |
| Model/LLM layer             | Start with a baseline and justify model size, latency and cost.            |
| Evaluation                  | Use offline metrics, golden examples and online feedback where possible.   |
| Monitoring                  | Track quality, latency, errors, drift, token usage and user feedback.      |
| Failure handling            | Use retries, fallbacks, abstention, human escalation and rollback.         |

## Interview explanation frame

- I would start fraud detection by clarifying the users, expected volume and risk level.
- I would define the data contract before choosing the model.
- I would build a baseline, measure it, then add complexity only if required.
- I would ship with logging, evaluation, monitoring and a rollback path.

## 6. Document Intelligence Extractor

Extract fields from documents with confidence and source evidence.

### High-level architecture



| Design area                 | Production decision                                                        |
|-----------------------------|----------------------------------------------------------------------------|
| Functional requirements     | Define user action, input format, expected output and acceptance criteria. |
| Non-functional requirements | Latency, throughput, availability, cost, security and compliance.          |
| Data layer                  | Validate schema, freshness, duplicates, access control and lineage.        |
| Model/LLM layer             | Start with a baseline and justify model size, latency and cost.            |
| Evaluation                  | Use offline metrics, golden examples and online feedback where possible.   |
| Monitoring                  | Track quality, latency, errors, drift, token usage and user feedback.      |
| Failure handling            | Use retries, fallbacks, abstention, human escalation and rollback.         |

### Interview explanation frame

- I would start document intelligence extractor by clarifying the users, expected volume and risk level.
- I would define the data contract before choosing the model.
- I would build a baseline, measure it, then add complexity only if required.
- I would ship with logging, evaluation, monitoring and a rollback path.



# 7. Basic RAG Chatbot

Answer from a document base with citations and retrieval evaluation.

## High-level architecture



| Design area                 | Production decision                                                        |
|-----------------------------|----------------------------------------------------------------------------|
| Functional requirements     | Define user action, input format, expected output and acceptance criteria. |
| Non-functional requirements | Latency, throughput, availability, cost, security and compliance.          |
| Data layer                  | Validate schema, freshness, duplicates, access control and lineage.        |
| Model/LLM layer             | Start with a baseline and justify model size, latency and cost.            |
| Evaluation                  | Use offline metrics, golden examples and online feedback where possible.   |
| Monitoring                  | Track quality, latency, errors, drift, token usage and user feedback.      |
| Failure handling            | Use retries, fallbacks, abstention, human escalation and rollback.         |

## Interview explanation frame

- I would start basic rag chatbot by clarifying the users, expected volume and risk level.
- I would define the data contract before choosing the model.
- I would build a baseline, measure it, then add complexity only if required.
- I would ship with logging, evaluation, monitoring and a rollback path.

# 8. Agentic RAG Workflow

Use an agent only where decisions or tool use are necessary.

## High-level architecture



| Design area                 | Production decision                                                        |
|-----------------------------|----------------------------------------------------------------------------|
| Functional requirements     | Define user action, input format, expected output and acceptance criteria. |
| Non-functional requirements | Latency, throughput, availability, cost, security and compliance.          |
| Data layer                  | Validate schema, freshness, duplicates, access control and lineage.        |
| Model/LLM layer             | Start with a baseline and justify model size, latency and cost.            |
| Evaluation                  | Use offline metrics, golden examples and online feedback where possible.   |
| Monitoring                  | Track quality, latency, errors, drift, token usage and user feedback.      |
| Failure handling            | Use retries, fallbacks, abstention, human escalation and rollback.         |

## Interview explanation frame

- I would start agentic rag workflow by clarifying the users, expected volume and risk level.
- I would define the data contract before choosing the model.
- I would build a baseline, measure it, then add complexity only if required.
- I would ship with logging, evaluation, monitoring and a rollback path.

# 9. Corrective RAG

Improve answer quality by checking retrieval sufficiency.

## High-level architecture



| Design area                 | Production decision                                                        |
|-----------------------------|----------------------------------------------------------------------------|
| Functional requirements     | Define user action, input format, expected output and acceptance criteria. |
| Non-functional requirements | Latency, throughput, availability, cost, security and compliance.          |
| Data layer                  | Validate schema, freshness, duplicates, access control and lineage.        |
| Model/LLM layer             | Start with a baseline and justify model size, latency and cost.            |
| Evaluation                  | Use offline metrics, golden examples and online feedback where possible.   |
| Monitoring                  | Track quality, latency, errors, drift, token usage and user feedback.      |
| Failure handling            | Use retries, fallbacks, abstention, human escalation and rollback.         |

## Interview explanation frame

- I would start corrective rag by clarifying the users, expected volume and risk level.
- I would define the data contract before choosing the model.
- I would build a baseline, measure it, then add complexity only if required.
- I would ship with logging, evaluation, monitoring and a rollback path.

# 10. GraphRAG

Use relationships to answer multi-hop questions.

## High-level architecture



| Design area                 | Production decision                                                        |
|-----------------------------|----------------------------------------------------------------------------|
| Functional requirements     | Define user action, input format, expected output and acceptance criteria. |
| Non-functional requirements | Latency, throughput, availability, cost, security and compliance.          |
| Data layer                  | Validate schema, freshness, duplicates, access control and lineage.        |
| Model/LLM layer             | Start with a baseline and justify model size, latency and cost.            |
| Evaluation                  | Use offline metrics, golden examples and online feedback where possible.   |
| Monitoring                  | Track quality, latency, errors, drift, token usage and user feedback.      |
| Failure handling            | Use retries, fallbacks, abstention, human escalation and rollback.         |

## Interview explanation frame

- I would start graphrag by clarifying the users, expected volume and risk level.
- I would define the data contract before choosing the model.
- I would build a baseline, measure it, then add complexity only if required.
- I would ship with logging, evaluation, monitoring and a rollback path.

# 11. SQL Analytics Agent

Answer structured-data questions with safe SQL generation.

## High-level architecture



| Design area                 | Production decision                                                        |
|-----------------------------|----------------------------------------------------------------------------|
| Functional requirements     | Define user action, input format, expected output and acceptance criteria. |
| Non-functional requirements | Latency, throughput, availability, cost, security and compliance.          |
| Data layer                  | Validate schema, freshness, duplicates, access control and lineage.        |
| Model/LLM layer             | Start with a baseline and justify model size, latency and cost.            |
| Evaluation                  | Use offline metrics, golden examples and online feedback where possible.   |
| Monitoring                  | Track quality, latency, errors, drift, token usage and user feedback.      |
| Failure handling            | Use retries, fallbacks, abstention, human escalation and rollback.         |

## Interview explanation frame

- I would start sql analytics agent by clarifying the users, expected volume and risk level.
- I would define the data contract before choosing the model.
- I would build a baseline, measure it, then add complexity only if required.
- I would ship with logging, evaluation, monitoring and a rollback path.

## 12. LLM Support Desk

Automate repetitive support while escalating uncertain cases.

### High-level architecture



| Design area                 | Production decision                                                        |
|-----------------------------|----------------------------------------------------------------------------|
| Functional requirements     | Define user action, input format, expected output and acceptance criteria. |
| Non-functional requirements | Latency, throughput, availability, cost, security and compliance.          |
| Data layer                  | Validate schema, freshness, duplicates, access control and lineage.        |
| Model/LLM layer             | Start with a baseline and justify model size, latency and cost.            |
| Evaluation                  | Use offline metrics, golden examples and online feedback where possible.   |
| Monitoring                  | Track quality, latency, errors, drift, token usage and user feedback.      |
| Failure handling            | Use retries, fallbacks, abstention, human escalation and rollback.         |

### Interview explanation frame

- I would start llm support desk by clarifying the users, expected volume and risk level.
- I would define the data contract before choosing the model.
- I would build a baseline, measure it, then add complexity only if required.
- I would ship with logging, evaluation, monitoring and a rollback path.

# 13. Cost-Aware LLM Router

Reduce cost by routing simple tasks away from expensive LLMs.

## High-level architecture



| Design area                 | Production decision                                                        |
|-----------------------------|----------------------------------------------------------------------------|
| Functional requirements     | Define user action, input format, expected output and acceptance criteria. |
| Non-functional requirements | Latency, throughput, availability, cost, security and compliance.          |
| Data layer                  | Validate schema, freshness, duplicates, access control and lineage.        |
| Model/LLM layer             | Start with a baseline and justify model size, latency and cost.            |
| Evaluation                  | Use offline metrics, golden examples and online feedback where possible.   |
| Monitoring                  | Track quality, latency, errors, drift, token usage and user feedback.      |
| Failure handling            | Use retries, fallbacks, abstention, human escalation and rollback.         |

## Interview explanation frame

- I would start cost-aware llm router by clarifying the users, expected volume and risk level.
- I would define the data contract before choosing the model.
- I would build a baseline, measure it, then add complexity only if required.
- I would ship with logging, evaluation, monitoring and a rollback path.

# 14. Local LLM Private RAG

Run sensitive workflows without sending data to external APIs.

## High-level architecture



| Design area                 | Production decision                                                        |
|-----------------------------|----------------------------------------------------------------------------|
| Functional requirements     | Define user action, input format, expected output and acceptance criteria. |
| Non-functional requirements | Latency, throughput, availability, cost, security and compliance.          |
| Data layer                  | Validate schema, freshness, duplicates, access control and lineage.        |
| Model/LLM layer             | Start with a baseline and justify model size, latency and cost.            |
| Evaluation                  | Use offline metrics, golden examples and online feedback where possible.   |
| Monitoring                  | Track quality, latency, errors, drift, token usage and user feedback.      |
| Failure handling            | Use retries, fallbacks, abstention, human escalation and rollback.         |

## Interview explanation frame

- I would start local llm private rag by clarifying the users, expected volume and risk level.
- I would define the data contract before choosing the model.
- I would build a baseline, measure it, then add complexity only if required.
- I would ship with logging, evaluation, monitoring and a rollback path.



# 15. Fine-Tuning Pipeline

Specialize model behavior with adapters and controlled evaluation.

## High-level architecture



| Design area                 | Production decision                                                        |
|-----------------------------|----------------------------------------------------------------------------|
| Functional requirements     | Define user action, input format, expected output and acceptance criteria. |
| Non-functional requirements | Latency, throughput, availability, cost, security and compliance.          |
| Data layer                  | Validate schema, freshness, duplicates, access control and lineage.        |
| Model/LLM layer             | Start with a baseline and justify model size, latency and cost.            |
| Evaluation                  | Use offline metrics, golden examples and online feedback where possible.   |
| Monitoring                  | Track quality, latency, errors, drift, token usage and user feedback.      |
| Failure handling            | Use retries, fallbacks, abstention, human escalation and rollback.         |

## Interview explanation frame

- I would start fine-tuning pipeline by clarifying the users, expected volume and risk level.
- I would define the data contract before choosing the model.
- I would build a baseline, measure it, then add complexity only if required.
- I would ship with logging, evaluation, monitoring and a rollback path.

# 16. LLM Evaluation Platform

Prevent prompt/model changes from breaking quality.

## High-level architecture



| Design area                 | Production decision                                                        |
|-----------------------------|----------------------------------------------------------------------------|
| Functional requirements     | Define user action, input format, expected output and acceptance criteria. |
| Non-functional requirements | Latency, throughput, availability, cost, security and compliance.          |
| Data layer                  | Validate schema, freshness, duplicates, access control and lineage.        |
| Model/LLM layer             | Start with a baseline and justify model size, latency and cost.            |
| Evaluation                  | Use offline metrics, golden examples and online feedback where possible.   |
| Monitoring                  | Track quality, latency, errors, drift, token usage and user feedback.      |
| Failure handling            | Use retries, fallbacks, abstention, human escalation and rollback.         |

## Interview explanation frame

- I would start llm evaluation platform by clarifying the users, expected volume and risk level.
- I would define the data contract before choosing the model.
- I would build a baseline, measure it, then add complexity only if required.
- I would ship with logging, evaluation, monitoring and a rollback path.

# 17. LLMOps Observability

Debug and monitor GenAI applications end to end.

## High-level architecture



| Design area                 | Production decision                                                        |
|-----------------------------|----------------------------------------------------------------------------|
| Functional requirements     | Define user action, input format, expected output and acceptance criteria. |
| Non-functional requirements | Latency, throughput, availability, cost, security and compliance.          |
| Data layer                  | Validate schema, freshness, duplicates, access control and lineage.        |
| Model/LLM layer             | Start with a baseline and justify model size, latency and cost.            |
| Evaluation                  | Use offline metrics, golden examples and online feedback where possible.   |
| Monitoring                  | Track quality, latency, errors, drift, token usage and user feedback.      |
| Failure handling            | Use retries, fallbacks, abstention, human escalation and rollback.         |

## Interview explanation frame

- I would start llmops observability by clarifying the users, expected volume and risk level.
- I would define the data contract before choosing the model.
- I would build a baseline, measure it, then add complexity only if required.
- I would ship with logging, evaluation, monitoring and a rollback path.

# 18. Feature Store Architecture

Keep training and serving features consistent.

## High-level architecture



| Design area                 | Production decision                                                        |
|-----------------------------|----------------------------------------------------------------------------|
| Functional requirements     | Define user action, input format, expected output and acceptance criteria. |
| Non-functional requirements | Latency, throughput, availability, cost, security and compliance.          |
| Data layer                  | Validate schema, freshness, duplicates, access control and lineage.        |
| Model/LLM layer             | Start with a baseline and justify model size, latency and cost.            |
| Evaluation                  | Use offline metrics, golden examples and online feedback where possible.   |
| Monitoring                  | Track quality, latency, errors, drift, token usage and user feedback.      |
| Failure handling            | Use retries, fallbacks, abstention, human escalation and rollback.         |

## Interview explanation frame

- I would start feature store architecture by clarifying the users, expected volume and risk level.
- I would define the data contract before choosing the model.
- I would build a baseline, measure it, then add complexity only if required.
- I would ship with logging, evaluation, monitoring and a rollback path.

# 19. Batch Scoring System

Score large datasets periodically with job-level monitoring.

## High-level architecture



| Design area                 | Production decision                                                        |
|-----------------------------|----------------------------------------------------------------------------|
| Functional requirements     | Define user action, input format, expected output and acceptance criteria. |
| Non-functional requirements | Latency, throughput, availability, cost, security and compliance.          |
| Data layer                  | Validate schema, freshness, duplicates, access control and lineage.        |
| Model/LLM layer             | Start with a baseline and justify model size, latency and cost.            |
| Evaluation                  | Use offline metrics, golden examples and online feedback where possible.   |
| Monitoring                  | Track quality, latency, errors, drift, token usage and user feedback.      |
| Failure handling            | Use retries, fallbacks, abstention, human escalation and rollback.         |

## Interview explanation frame

- I would start batch scoring system by clarifying the users, expected volume and risk level.
- I would define the data contract before choosing the model.
- I would build a baseline, measure it, then add complexity only if required.
- I would ship with logging, evaluation, monitoring and a rollback path.

## 20. Real-Time Inference System

Serve predictions under latency and reliability constraints.

### High-level architecture



| Design area                 | Production decision                                                        |
|-----------------------------|----------------------------------------------------------------------------|
| Functional requirements     | Define user action, input format, expected output and acceptance criteria. |
| Non-functional requirements | Latency, throughput, availability, cost, security and compliance.          |
| Data layer                  | Validate schema, freshness, duplicates, access control and lineage.        |
| Model/LLM layer             | Start with a baseline and justify model size, latency and cost.            |
| Evaluation                  | Use offline metrics, golden examples and online feedback where possible.   |
| Monitoring                  | Track quality, latency, errors, drift, token usage and user feedback.      |
| Failure handling            | Use retries, fallbacks, abstention, human escalation and rollback.         |

### Interview explanation frame

- I would start real-time inference system by clarifying the users, expected volume and risk level.
- I would define the data contract before choosing the model.
- I would build a baseline, measure it, then add complexity only if required.
- I would ship with logging, evaluation, monitoring and a rollback path.

# 21. Model Monitoring System

Observe production model behavior and decide when to act.

## High-level architecture



| Design area                 | Production decision                                                        |
|-----------------------------|----------------------------------------------------------------------------|
| Functional requirements     | Define user action, input format, expected output and acceptance criteria. |
| Non-functional requirements | Latency, throughput, availability, cost, security and compliance.          |
| Data layer                  | Validate schema, freshness, duplicates, access control and lineage.        |
| Model/LLM layer             | Start with a baseline and justify model size, latency and cost.            |
| Evaluation                  | Use offline metrics, golden examples and online feedback where possible.   |
| Monitoring                  | Track quality, latency, errors, drift, token usage and user feedback.      |
| Failure handling            | Use retries, fallbacks, abstention, human escalation and rollback.         |

## Interview explanation frame

- I would start model monitoring system by clarifying the users, expected volume and risk level.
- I would define the data contract before choosing the model.
- I would build a baseline, measure it, then add complexity only if required.
- I would ship with logging, evaluation, monitoring and a rollback path.

## 22. Data Quality Gate

Stop bad data before it contaminates models or dashboards.

### High-level architecture



| Design area                 | Production decision                                                        |
|-----------------------------|----------------------------------------------------------------------------|
| Functional requirements     | Define user action, input format, expected output and acceptance criteria. |
| Non-functional requirements | Latency, throughput, availability, cost, security and compliance.          |
| Data layer                  | Validate schema, freshness, duplicates, access control and lineage.        |
| Model/LLM layer             | Start with a baseline and justify model size, latency and cost.            |
| Evaluation                  | Use offline metrics, golden examples and online feedback where possible.   |
| Monitoring                  | Track quality, latency, errors, drift, token usage and user feedback.      |
| Failure handling            | Use retries, fallbacks, abstention, human escalation and rollback.         |

### Interview explanation frame

- I would start data quality gate by clarifying the users, expected volume and risk level.
- I would define the data contract before choosing the model.
- I would build a baseline, measure it, then add complexity only if required.
- I would ship with logging, evaluation, monitoring and a rollback path.



## 23. Human-in-the-Loop Review

Use human approval for risky or low-confidence outputs.

### High-level architecture



| Design area                 | Production decision                                                        |
|-----------------------------|----------------------------------------------------------------------------|
| Functional requirements     | Define user action, input format, expected output and acceptance criteria. |
| Non-functional requirements | Latency, throughput, availability, cost, security and compliance.          |
| Data layer                  | Validate schema, freshness, duplicates, access control and lineage.        |
| Model/LLM layer             | Start with a baseline and justify model size, latency and cost.            |
| Evaluation                  | Use offline metrics, golden examples and online feedback where possible.   |
| Monitoring                  | Track quality, latency, errors, drift, token usage and user feedback.      |
| Failure handling            | Use retries, fallbacks, abstention, human escalation and rollback.         |

### Interview explanation frame

- I would start human-in-the-loop review by clarifying the users, expected volume and risk level.
- I would define the data contract before choosing the model.
- I would build a baseline, measure it, then add complexity only if required.
- I would ship with logging, evaluation, monitoring and a rollback path.

# 24. AI Governance and Audit

Provide traceability for enterprise and regulated use cases.

## High-level architecture



| Design area                 | Production decision                                                        |
|-----------------------------|----------------------------------------------------------------------------|
| Functional requirements     | Define user action, input format, expected output and acceptance criteria. |
| Non-functional requirements | Latency, throughput, availability, cost, security and compliance.          |
| Data layer                  | Validate schema, freshness, duplicates, access control and lineage.        |
| Model/LLM layer             | Start with a baseline and justify model size, latency and cost.            |
| Evaluation                  | Use offline metrics, golden examples and online feedback where possible.   |
| Monitoring                  | Track quality, latency, errors, drift, token usage and user feedback.      |
| Failure handling            | Use retries, fallbacks, abstention, human escalation and rollback.         |

## Interview explanation frame

- I would start ai governance and audit by clarifying the users, expected volume and risk level.
- I would define the data contract before choosing the model.
- I would build a baseline, measure it, then add complexity only if required.
- I would ship with logging, evaluation, monitoring and a rollback path.

# 25. Multimodal RAG

Answer questions over documents with tables, images and layout.

## High-level architecture



| Design area                 | Production decision                                                        |
|-----------------------------|----------------------------------------------------------------------------|
| Functional requirements     | Define user action, input format, expected output and acceptance criteria. |
| Non-functional requirements | Latency, throughput, availability, cost, security and compliance.          |
| Data layer                  | Validate schema, freshness, duplicates, access control and lineage.        |
| Model/LLM layer             | Start with a baseline and justify model size, latency and cost.            |
| Evaluation                  | Use offline metrics, golden examples and online feedback where possible.   |
| Monitoring                  | Track quality, latency, errors, drift, token usage and user feedback.      |
| Failure handling            | Use retries, fallbacks, abstention, human escalation and rollback.         |

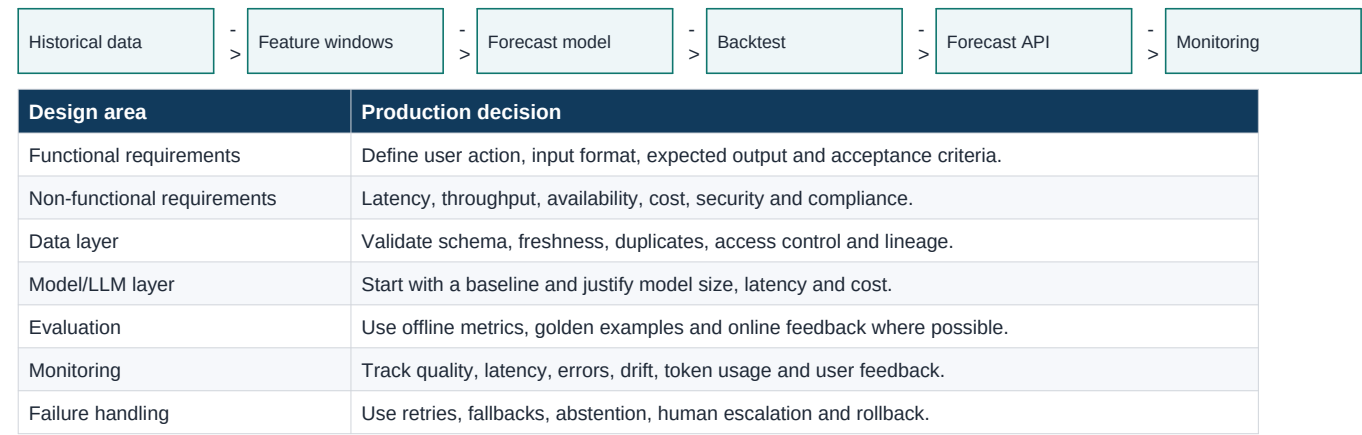
## Interview explanation frame

- I would start multimodal rag by clarifying the users, expected volume and risk level.
- I would define the data contract before choosing the model.
- I would build a baseline, measure it, then add complexity only if required.
- I would ship with logging, evaluation, monitoring and a rollback path.

## 26. Time-Series Forecasting

Forecast demand, volume, revenue or workload over time.

### High-level architecture



### Interview explanation frame

- I would start time-series forecasting by clarifying the users, expected volume and risk level.
- I would define the data contract before choosing the model.
- I would build a baseline, measure it, then add complexity only if required.
- I would ship with logging, evaluation, monitoring and a rollback path.

## 27. Anomaly Detection Pipeline

Detect unusual patterns with feedback-driven thresholds.

### High-level architecture



| Design area                 | Production decision                                                        |
|-----------------------------|----------------------------------------------------------------------------|
| Functional requirements     | Define user action, input format, expected output and acceptance criteria. |
| Non-functional requirements | Latency, throughput, availability, cost, security and compliance.          |
| Data layer                  | Validate schema, freshness, duplicates, access control and lineage.        |
| Model/LLM layer             | Start with a baseline and justify model size, latency and cost.            |
| Evaluation                  | Use offline metrics, golden examples and online feedback where possible.   |
| Monitoring                  | Track quality, latency, errors, drift, token usage and user feedback.      |
| Failure handling            | Use retries, fallbacks, abstention, human escalation and rollback.         |

### Interview explanation frame

- I would start anomaly detection pipeline by clarifying the users, expected volume and risk level.
- I would define the data contract before choosing the model.
- I would build a baseline, measure it, then add complexity only if required.
- I would ship with logging, evaluation, monitoring and a rollback path.

## 28. A/B Testing for ML

Measure model impact on real business metrics safely.

### High-level architecture



| Design area                 | Production decision                                                        |
|-----------------------------|----------------------------------------------------------------------------|
| Functional requirements     | Define user action, input format, expected output and acceptance criteria. |
| Non-functional requirements | Latency, throughput, availability, cost, security and compliance.          |
| Data layer                  | Validate schema, freshness, duplicates, access control and lineage.        |
| Model/LLM layer             | Start with a baseline and justify model size, latency and cost.            |
| Evaluation                  | Use offline metrics, golden examples and online feedback where possible.   |
| Monitoring                  | Track quality, latency, errors, drift, token usage and user feedback.      |
| Failure handling            | Use retries, fallbacks, abstention, human escalation and rollback.         |

### Interview explanation frame

- I would start a/b testing for ml by clarifying the users, expected volume and risk level.
- I would define the data contract before choosing the model.
- I would build a baseline, measure it, then add complexity only if required.
- I would ship with logging, evaluation, monitoring and a rollback path.

# 29. MLOps CI/CD

Automate safe model delivery from code to production.

## High-level architecture



| Design area                 | Production decision                                                        |
|-----------------------------|----------------------------------------------------------------------------|
| Functional requirements     | Define user action, input format, expected output and acceptance criteria. |
| Non-functional requirements | Latency, throughput, availability, cost, security and compliance.          |
| Data layer                  | Validate schema, freshness, duplicates, access control and lineage.        |
| Model/LLM layer             | Start with a baseline and justify model size, latency and cost.            |
| Evaluation                  | Use offline metrics, golden examples and online feedback where possible.   |
| Monitoring                  | Track quality, latency, errors, drift, token usage and user feedback.      |
| Failure handling            | Use retries, fallbacks, abstention, human escalation and rollback.         |

## Interview explanation frame

- I would start mlops ci/cd by clarifying the users, expected volume and risk level.
- I would define the data contract before choosing the model.
- I would build a baseline, measure it, then add complexity only if required.
- I would ship with logging, evaluation, monitoring and a rollback path.

# 30. RAG Security Gateway

Prevent data leakage and unsafe actions in RAG systems.

## High-level architecture



| Design area                 | Production decision                                                        |
|-----------------------------|----------------------------------------------------------------------------|
| Functional requirements     | Define user action, input format, expected output and acceptance criteria. |
| Non-functional requirements | Latency, throughput, availability, cost, security and compliance.          |
| Data layer                  | Validate schema, freshness, duplicates, access control and lineage.        |
| Model/LLM layer             | Start with a baseline and justify model size, latency and cost.            |
| Evaluation                  | Use offline metrics, golden examples and online feedback where possible.   |
| Monitoring                  | Track quality, latency, errors, drift, token usage and user feedback.      |
| Failure handling            | Use retries, fallbacks, abstention, human escalation and rollback.         |

## Interview explanation frame

- I would start rag security gateway by clarifying the users, expected volume and risk level.
- I would define the data contract before choosing the model.
- I would build a baseline, measure it, then add complexity only if required.
- I would ship with logging, evaluation, monitoring and a rollback path.



# 31. Agent Tool Execution Sandbox

Control what agents can do when connected to tools.

## High-level architecture



| Design area                 | Production decision                                                        |
|-----------------------------|----------------------------------------------------------------------------|
| Functional requirements     | Define user action, input format, expected output and acceptance criteria. |
| Non-functional requirements | Latency, throughput, availability, cost, security and compliance.          |
| Data layer                  | Validate schema, freshness, duplicates, access control and lineage.        |
| Model/LLM layer             | Start with a baseline and justify model size, latency and cost.            |
| Evaluation                  | Use offline metrics, golden examples and online feedback where possible.   |
| Monitoring                  | Track quality, latency, errors, drift, token usage and user feedback.      |
| Failure handling            | Use retries, fallbacks, abstention, human escalation and rollback.         |

## Interview explanation frame

- I would start agent tool execution sandbox by clarifying the users, expected volume and risk level.
- I would define the data contract before choosing the model.
- I would build a baseline, measure it, then add complexity only if required.
- I would ship with logging, evaluation, monitoring and a rollback path.

# 32. Enterprise AI Analytics App

Turn operational data into dynamic analytics with AI explanations.

## High-level architecture



| Design area                 | Production decision                                                        |
|-----------------------------|----------------------------------------------------------------------------|
| Functional requirements     | Define user action, input format, expected output and acceptance criteria. |
| Non-functional requirements | Latency, throughput, availability, cost, security and compliance.          |
| Data layer                  | Validate schema, freshness, duplicates, access control and lineage.        |
| Model/LLM layer             | Start with a baseline and justify model size, latency and cost.            |
| Evaluation                  | Use offline metrics, golden examples and online feedback where possible.   |
| Monitoring                  | Track quality, latency, errors, drift, token usage and user feedback.      |
| Failure handling            | Use retries, fallbacks, abstention, human escalation and rollback.         |

## Interview explanation frame

- I would start enterprise ai analytics app by clarifying the users, expected volume and risk level.
- I would define the data contract before choosing the model.
- I would build a baseline, measure it, then add complexity only if required.
- I would ship with logging, evaluation, monitoring and a rollback path.